LOCAL-AI LLM GETTING-STARTED PRIVACY

What Is Local AI? A Complete Guide

Local AI means running AI models on your own hardware, with local data processing, no per-token API fees, and the ability to work offline.

Published April 8, 2026 · By Local LLM Team

Local AI is the practice of running artificial intelligence models — large language models (LLMs), image generators, voice synthesizers, embedding models, and more — entirely on hardware you own and control. Instead of sending your prompts, documents, and data to a cloud provider like OpenAI, Google, or Anthropic, local AI processes everything on your desktop, laptop, phone, or private server. Configured for local-only use, your data stays on your machine. There are no API fees, no usage limits, no internet requirement once models are downloaded, and no third-party terms of service governing what you can do. Local AI gives you the capabilities of modern AI with the privacy and autonomy of software you run yourself.

The local AI movement has accelerated dramatically since 2023. Open-weight models from Meta (Llama), Mistral, Google (Gemma), DeepSeek, Microsoft (Phi), and Qwen now rival proprietary models on many benchmarks. Inference engines like llama.cpp, Ollama, and vLLM have made running these models accessible to anyone with a modern computer. What once required a data center can now run on a laptop.

This guide is the definitive resource for understanding local AI — what it is, how it works, what you can do with it, and how to get started.

What is local AI?

Local AI refers to any artificial intelligence system that runs entirely on local hardware — meaning hardware that you physically own, control, or operate — rather than relying on cloud-hosted services. The term encompasses a broad range of AI capabilities:

- **Large language models (LLMs)** for chat, writing, coding, reasoning, and analysis
- **Vision models** for image understanding, OCR, and visual question-answering
- **Image generation models** like Stable Diffusion for creating artwork and graphics
- **Embedding models** for semantic search, retrieval-augmented generation (RAG), and classification
- **Voice models** for speech-to-text (Whisper) and text-to-speech synthesis
- **Code models** fine-tuned for programming tasks, code completion, and debugging

The defining characteristic of local AI is **data locality**. When you type a prompt into ChatGPT, your text travels over the internet to OpenAI's servers, is processed there, and the response is sent back. With local AI, the model weights live on your machine, inference happens on your CPU or GPU, and, with a local-only configuration, the data never touches an external network. This distinction has profound implications for privacy, cost, latency, and control.

Local AI is sometimes called "on-device AI," "on-premise AI," "edge AI," or "self-hosted AI." While there are subtle differences between these terms — edge AI often refers to IoT and embedded devices, on-premise typically implies enterprise servers — they all share the core principle: the AI runs where you are, not in someone else's cloud.

The rise of open-weight models

Local AI became practical because of the open-weight model revolution. In February 2023, Meta released LLaMA, and within weeks the community had it running on consumer hardware. Since then, the pace has been relentless:

- **Meta Llama 3.2** (2024-2025): Available in 1B, 3B, 8B, 70B, and 405B parameter sizes, with multimodal variants. The 8B model rivals GPT-3.5 on most tasks.

- **DeepSeek-R1** (2025): A reasoning-focused model that matches GPT-4 on mathematics, coding, and logical reasoning benchmarks.
- **Mistral and Mixtral**: Efficient models with strong multilingual capabilities and mixture-of-experts architectures.
- **Google Gemma 2**: Compact, high-quality models at 2B, 9B, and 27B sizes.
- **Microsoft Phi-3/Phi-4**: Small language models (SLMs) that punch well above their weight class, achieving strong results at 3.8B and 14B parameters.
- **Qwen 2.5**: A comprehensive family from Alibaba covering language, code, math, and vision.

These models are released under permissive licenses that allow commercial use, fine-tuning, and redistribution. They are the foundation of the local AI ecosystem.

How does local AI work?

Running AI locally involves three core components: a **model**, an **inference engine**, and **hardware**. Understanding how they interact is essential to getting the best performance from your local setup.

Models and model formats

AI models are massive files containing billions of numerical parameters — the "weights" — that encode the model's learned knowledge. A raw model in full precision (FP16) requires approximately 2 bytes per parameter. This means a 7B parameter model needs roughly 14 GB just for the weights, and a 70B model requires around 140 GB.

To make models practical on consumer hardware, the community developed **quantization** — a technique that reduces the precision of model weights from 16-bit floating point to 8-bit, 4-bit, or even 2-bit integers. The most popular quantization format is **GGUF** (GPT-Generated Unified Format), created by the llama.cpp project. A 7B model quantized to 4-bit (Q4_K_M) shrinks to about 4 GB with minimal quality loss.

Common model formats you will encounter:

Format	Description	Used By
GGUF	Quantized format for CPU and GPU inference	llama.cpp, Ollama, LM Studio, GPT4All
SafeTensors	Standard format for full-precision and GPTQ/AWQ quantized models	Hugging Face, vLLM, TGI
GPTQ	GPU-optimized quantization format	ExLlama, vLLM, AutoGPTQ
AWQ	Activation-aware quantization, efficient on modern GPUs	vLLM, TGI
EXL2	Advanced GPU quantization with variable bit rates	ExLlamaV2

Inference engines

An inference engine is the software that loads a model into memory and runs it — accepting your prompt as input and generating text (or images, or embeddings) as output. The inference engine handles tokenization, attention computation, KV-cache management, and sampling. Popular engines include:

- **llama.cpp**: The foundational C/C++ inference engine. Runs GGUF models on CPU, CUDA GPUs, Metal (Apple), ROCm (AMD), and Vulkan. Extremely portable and efficient.
- **Ollama**: A user-friendly wrapper around llama.cpp that provides a one-command install, a model registry, and an OpenAI-compatible API. The easiest way to get started.
- **vLLM**: A high-performance engine optimized for GPU throughput with PagedAttention, continuous batching, and tensor parallelism. Ideal for serving models to multiple users.
- **LM Studio**: A desktop application with a graphical interface for downloading, running, and chatting with models. Cross-platform (Mac, Windows, Linux).

- **ExLlamaV2**: A highly optimized GPU inference engine for EXL2 and GPTQ models, known for fast token generation speeds.
- **TGI (Text Generation Inference)**: Hugging Face's production inference server with support for SafeTensors, GPTQ, and AWQ models.
- **MLX**: Apple's machine learning framework optimized specifically for Apple Silicon, offering native Metal acceleration.
- **MLC LLM**: A universal deployment solution that compiles models for CPUs, GPUs, phones, and browsers using Apache TVM.

The inference process

When you send a prompt to a local AI, here is what happens:

1. **Tokenization**: Your text is converted into tokens — numerical IDs that represent words, subwords, or characters. A typical prompt of 500 words becomes roughly 600-700 tokens.
2. **Prefill (prompt processing)**: The model processes all input tokens in parallel, computing attention across the entire prompt. This is the most compute-intensive step and is primarily limited by memory bandwidth.
3. **Decode (token generation)**: The model generates output tokens one at a time, each depending on all previous tokens. Speed is measured in tokens per second (tok/s). A good local setup produces 20-80 tok/s depending on model size and hardware.
4. **Sampling**: At each step, the model produces probabilities for all possible next tokens. Sampling parameters (temperature, top-p, top-k) control how the next token is selected.
5. **KV-cache management**: The model stores key-value pairs from previous tokens to avoid recomputing them. This cache grows with context length and is a major consumer of memory.

Hardware: what runs the model

Local AI performance depends on three hardware factors:

- **Memory (RAM or VRAM)**: The model weights must fit in memory. This is the primary constraint. If a model is 8 GB, you need at least 8 GB of available VRAM (for GPU inference) or RAM (for CPU inference), plus additional memory for the KV-cache and operating system overhead.
- **Memory bandwidth**: Token generation speed is limited by how fast you can read model weights from memory. NVIDIA GPUs offer 300-1000+ GB/s bandwidth. Apple Silicon unified memory provides 100-800 GB/s. DDR5 RAM offers 50-80 GB/s. This is why GPUs generate tokens faster than CPUs.
- **Compute (FLOPS)**: Important during the prefill phase. Modern GPUs have massive parallel compute capabilities that accelerate prompt processing.

For a detailed hardware buying guide, see our hardware requirements guide.

How does local AI compare to cloud AI?

The choice between local and cloud AI involves trade-offs across privacy, cost, performance, flexibility, and convenience. Here is a comprehensive comparison:

Dimension	Local AI	Cloud AI
Data privacy	Data stays on your device when tools are configured for local-only use	Data sent to third-party servers; subject to provider policies
Cost model	Upfront hardware investment; no per-query fees (power and wear still apply)	Pay-per-token or subscription; costs scale with usage
Latency	No network round-trip; time-to-first-token depends on hardware and prompt length; typically 20-80 tok/s for mid-size models	Network latency + queue time; can be faster for large models
Offline access	Works offline once models are downloaded (excluding web/cloud features)	Requires internet connection

Dimension	Local AI	Cloud AI
Model selection	Any open-weight model; full control over version and quantization	Limited to provider's model catalog
Customization	Fine-tune, merge, quantize, and modify freely	Limited fine-tuning options; no weight access
Maximum model size	Limited by your hardware (typically up to 70B on consumer GPUs)	Access to the largest models (GPT-4, Claude, Gemini Ultra)
Setup complexity	Requires initial hardware and software setup	Sign up and get an API key
Maintenance	You manage updates, hardware, and troubleshooting	Provider handles everything
Scalability	Limited by your hardware; adding capacity requires buying more	Scales elastically with demand
Compliance	Full control over data residency and processing	May not meet data sovereignty requirements
Censorship/ filtering	You control content filtering	Provider-imposed content policies
Uptime	Depends on your hardware reliability	Enterprise SLAs (99.9% +)
Frontier capabilities	Lags behind proprietary models by weeks to months	Access to the latest capabilities immediately

The key insight is that **local and cloud AI are not mutually exclusive**. Many users and organizations adopt a hybrid approach: local AI for sensitive data, routine tasks, and high-volume workloads; cloud AI for tasks requiring frontier model capabilities or occasional complex reasoning.

For a deeper analysis, see our Local AI vs Cloud AI comparison.

What is the local AI stack?

A complete local AI setup has three layers, each with multiple options. Think of it like a web development stack — you choose a database, a backend, and a frontend. The local AI stack works the same way.

Layer 1: Inference engine

The inference engine is the foundation. It loads and runs the model. Your choice of engine determines which model formats you can use, which hardware is supported, and what performance you will get.

For beginners: Start with **Ollama**. One command to install, one command to download and run a model. It exposes an OpenAI-compatible API that most tools can connect to.

```
# Install Ollama
curl -fsSL https://ollama.ai/install.sh | sh

# Run a model
ollama run llama3.2
```

For power users: **llama.cpp** (via its `llama-server` binary) gives you full control over quantization, context length, GPU layer offloading, and batch size. **vLLM** is the best choice for high-throughput serving with multiple concurrent users.

For Mac users: **MLX** provides native Apple Silicon optimization with excellent performance on M1-M4 chips.

Layer 2: User interface

A UI layer provides a chat interface, conversation management, model switching, and often additional features like RAG, web search, and image generation.

- **Open WebUI**: The most popular open-source chat interface. Feature-rich, supports multiple backends, has RAG built in, and offers multi-user support. Connects to Ollama or any OpenAI-compatible API.
- **LM Studio**: A polished desktop app that bundles its own inference engine. Great for users who prefer a graphical experience for downloading and managing models.
- **GPT4All**: A desktop application focused on simplicity and privacy. Includes its own inference engine and model catalog.
- **Jan**: An open-source desktop app built on Electron with a clean interface and local-first architecture.
- **Llamafu**: A lightweight, fast UI designed specifically for Ollama, with a focus on speed and simplicity.
- **SillyTavern**: A UI oriented toward character-based roleplay and creative writing, popular in the hobbyist community.
- **text-generation-webui (oobabooga)**: A Gradio-based web interface with extensive model support and advanced parameter tuning.

Layer 3: Application framework

For building applications beyond simple chat, frameworks provide tools for RAG pipelines, agent workflows, function calling, and integration with external services.

- **LangChain / LangGraph**: The most comprehensive framework for building LLM applications. Supports chains, agents, RAG, tool use, and memory. Works with any OpenAI-compatible API (including Ollama).
- **LlamaIndex**: Focused on data ingestion and RAG. Excellent for building search and Q&A systems over your own documents.
- **Haystack**: An end-to-end NLP framework from deepset, designed for production RAG pipelines.
- **Semantic Kernel**: Microsoft's SDK for integrating LLMs into applications, with strong .NET and Python support.
- **CrewAI**: A framework for orchestrating multiple AI agents that collaborate on complex tasks.
- **Autogen**: Microsoft's framework for multi-agent conversations and automated workflows.

What can you do with local AI?

Local AI is not just a privacy-preserving alternative to ChatGPT. It enables use cases that are impractical or impossible with cloud services.

Chat and conversation

The most straightforward use case. Run a model locally and chat with it through a UI like Open WebUI or LM Studio. You get a private, always-available assistant that knows nothing about your conversations except what you tell it in the current session. Models like Llama 3.2 8B provide excellent conversational quality for general-purpose chat.

Coding assistance

Local code models like DeepSeek-Coder, CodeLlama, Qwen2.5-Coder, and StarCoder2 provide IDE-integrated code completion, generation, explanation, and debugging. Tools like Continue.dev connect directly to Ollama to provide a Copilot-like experience with zero cloud dependency. Your proprietary codebase stays entirely local.

Retrieval-augmented generation (RAG)

RAG combines a language model with a knowledge base to answer questions grounded in your own documents. Running RAG locally means your documents — contracts, medical records, financial reports, proprietary research — can stay within your infrastructure, provided embeddings, retrieval, and generation all run locally. A typical local RAG stack includes:

1. An **embedding model** (e.g., `nomic-embed-text` via Ollama) to convert

documents into vectors
2. A **vector database** (e.g., ChromaDB, Qdrant, Milvus) to store and search those vectors
3. A **language model** to generate answers using retrieved context

Voice and speech

Whisper (from OpenAI, ironically) runs locally via whisper.cpp and provides state-of-the-art speech-to-text. Combined with local text-to-speech models like Piper, Bark, or XTTS, you can build fully local voice assistants. The Whisper large-v3 model runs in real-time on a modern GPU.

Image generation

Stable Diffusion (via AUTOMATIC1111, ComfyUI, or Fooocus) generates images locally with full control over prompts, seeds, and model fine-tunes. SDXL and Stable Diffusion 3 produce high-quality images on a GPU with 8 GB or more VRAM. **Flux** models from Black Forest Labs represent the latest generation of local image generation.

Vision and multimodal

Models like LLaVA, Llama 3.2 Vision, and Qwen-VL can analyze images locally. Use cases include document OCR, screenshot analysis, image description for accessibility, and visual question-answering. Run them through Ollama with `ollama run llava` or `ollama run llama3.2-vision`.

Enterprise and compliance

Organizations in healthcare, finance, legal, and government face strict data regulations (HIPAA, GDPR, SOX, ITAR). Local deployment can make compliance much easier, though it does not make it automatic. Sensitive data — patient records, financial statements, classified documents — can be processed by AI while staying within the organization's infrastructure, alongside the access controls, logging, and retention policies the regulations require.

Automation and agents

Local AI agents can automate repetitive tasks: summarizing emails, classifying documents, extracting data from forms, generating reports, monitoring log files, and triaging support tickets. Because local inference has no per-call fees, you can run these automations continuously at the cost of power and capacity rather than API bills.

What hardware do you need for local AI?

Hardware requirements depend entirely on the models you want to run. Here is a quick overview:

Use Case	Model Size	Minimum RAM/VRAM	Example Hardware
Basic chat, simple tasks	1B-3B	4-8 GB RAM	Any modern laptop or desktop
Good general assistant	7B-8B	8-16 GB RAM or 6-8 GB VRAM	M1/M2 MacBook, RTX 3060
High-quality chat, coding	13B-14B	16-24 GB RAM or 10-12 GB VRAM	M2 Pro Mac, RTX 3090
Near-frontier quality	30B-34B	32-48 GB RAM or 24 GB VRAM	M2 Max/Ultra Mac, RTX 4090
Frontier-class	70B	48-64 GB RAM or 2x24 GB VRAM	M3 Ultra Mac, dual RTX 4090
Maximum capability	70B+ (405B)	128 GB+ RAM or multi-GPU	Mac Studio Ultra, 4-8x GPU server

Key takeaway: You do not need a $2,000 GPU to get started. A 7B-8B model runs

well on most computers made after 2020, including laptops. Apple Silicon Macs are particularly efficient because their unified memory architecture allows the CPU and GPU to share the same pool of fast memory — a Mac with 32 GB of unified memory can run a 30B model comfortably.

For detailed hardware recommendations, benchmarks, and buying advice, see our hardware requirements guide.

How do you get started with local AI?

Getting started takes five minutes with the right tools. Here is the quickest path:

Step 1: Install Ollama

Ollama is a free, open-source inference engine that works on macOS, Linux, and Windows. Install it with a single command:

```
# macOS / Linux
curl -fsSL https://ollama.ai/install.sh | sh

# Windows: download from https://ollama.ai
```

Step 2: Run your first model

Pull and run a model in one command:

```
# A great general-purpose model
ollama run llama3.2

# A small, fast model for testing
ollama run phi3

# A coding-focused model
ollama run deepseek-coder-v2
```

The first run downloads the model (a few GB). Subsequent runs start instantly.

Step 3: Add a user interface (optional)

For a richer experience with conversation history, model switching, and advanced features:

```
# Open WebUI (requires Docker)
docker run -d -p 3000:8080 \
  --add-host=host.docker.internal:host-gateway \
  -v open-webui:/app/backend/data \
  --name open-webui \
  ghcr.io/open-webui/open-webui:main
```

Or download LM Studio for a native desktop experience.

Step 4: Explore and experiment

Once you have a model running, try different tasks:

- Ask it to explain code, write functions, or debug errors
- Have it summarize documents or articles
- Use it for brainstorming and creative writing
- Build a RAG pipeline over your own documents using LangChain

For a detailed walkthrough, see our quickstart guide.

What tools make up the local AI ecosystem?

The local AI ecosystem is rich, rapidly evolving, and almost entirely open source. Here are the most important projects and tools:

Inference engines

- **llama.cpp**: The bedrock of the local AI movement. Written in C/C++, it runs

GGUF-quantized models on nearly any hardware. Created by Georgi Gerganov, it has become one of the most important open-source projects in AI.

- **Ollama**: Built on llama.cpp, Ollama provides a polished CLI and API experience. Its model registry makes downloading models as easy as pulling Docker images. The de facto standard for getting started with local AI.
- **vLLM**: The gold standard for high-throughput GPU inference. Features PagedAttention for efficient memory management, continuous batching, and tensor parallelism across multiple GPUs.
- **MLX**: Apple's machine learning framework, designed from the ground up for Apple Silicon. Offers excellent performance on M-series chips with a NumPy-like API.
- **ExLlamaV2**: A specialized GPU inference engine that achieves some of the fastest token generation speeds, particularly with EXL2 quantized models.

User interfaces

- **Open WebUI**: The most feature-complete chat interface. Supports RAG, web search, model management, multi-user auth, prompt templates, and a plugin system. Self-hosted via Docker.
- **LM Studio**: A beautifully designed desktop application for Mac, Windows, and Linux. Includes model discovery, download management, a chat interface, and a built-in server with an OpenAI-compatible API.
- **GPT4All**: A desktop app by Nomic AI focused on simplicity. Includes a built-in inference engine and curated model catalog. Great for non-technical users.
- **Llamafu**: A fast, lightweight chat UI built specifically for Ollama. Focuses on speed, simplicity, and a clean interface without the overhead of larger platforms.
- **Mullama**: A multi-provider interface that can connect to multiple Ollama instances and other backends simultaneously.

Frameworks and libraries

- **LangChain**: The Swiss Army knife of LLM application development. Provides abstractions for chains, agents, retrieval, memory, and tool use.
- **LlamaIndex**: Specializes in connecting LLMs to data. Excellent for building RAG systems, knowledge graphs, and document Q&A applications.
- **Haystack**: A production-ready NLP framework for building search and RAG pipelines with a pipeline-as-code approach.

Model sources

- **Hugging Face**: The central hub for open-weight models. Hosts model weights, datasets, and documentation. Most models are available in multiple formats and quantizations.
- **Ollama Model Library**: A curated registry of pre-packaged models ready for one-command download. Covers the most popular models in optimized GGUF formats.
- **TheBloke (Tom Jobbins)**: A prolific quantizer who has converted hundreds of models to GGUF, GPTQ, and AWQ formats, making them accessible to the community.

When should you choose local AI vs cloud AI?

The decision depends on your specific needs, technical capabilities, and constraints. Here are clear guidelines:

Choose local AI when:

- **Privacy is non-negotiable**: Medical records, legal documents, financial data, personal journals, proprietary code — anything you would not paste into a web form.
- **Cost predictability matters**: If you are making thousands of API calls per day, a one-time hardware investment pays for itself quickly. See our cost analysis in "Why Run AI Locally?".
- **You need offline access**: Airplanes, secure facilities, remote locations, or simply unreliable internet. Local AI works without a network connection once models and dependencies are downloaded.
- **Compliance requires it**: HIPAA, GDPR, ITAR, CJIS, and other regulations may prohibit sending data to third-party cloud providers.

- **You want full control**: Choose any model, any version, any quantization. Fine-tune on your own data. No content filters you did not choose. No sudden model deprecations.
- **Latency is critical**: For real-time applications, eliminating network round-trips can be the difference between usable and unusable. Local inference avoids network round-trips, though prompt processing still takes time.
- **High-volume, repetitive tasks**: Document processing, log analysis, data extraction, classification — tasks where you are running the same model thousands of times per day. Low marginal cost can make automation economical.

Choose cloud AI when:

- **You need the largest models**: GPT-4, Claude 3.5 Opus, and Gemini Ultra are currently only available via cloud APIs. These models outperform open-weight alternatives on the most complex reasoning tasks.
- **Usage is low or unpredictable**: If you make a few dozen queries per week, cloud APIs are cheaper than buying dedicated hardware.
- **You need zero setup**: Cloud APIs require only an API key. No hardware selection, no model management, no troubleshooting driver issues.
- **You want the latest capabilities immediately**: Proprietary models often lead open-weight models by weeks or months on new capabilities like tool use, long context, and multimodal understanding.
- **You need enterprise support**: Cloud providers offer SLAs, dedicated support, and compliance certifications that are harder to replicate with self-hosted infrastructure.

The hybrid approach

Many organizations and power users adopt a hybrid strategy:

1. **Local AI** handles sensitive data processing, routine tasks, development and testing, and high-volume workloads.
2. **Cloud AI** handles complex reasoning tasks, frontier model capabilities, and overflow during peak demand.
3. **Routing logic** (often implemented with a framework like LangChain) automatically directs each query to the appropriate backend based on sensitivity, complexity, and cost.

This hybrid model gives you the best of both worlds — privacy and cost efficiency where it matters, frontier capabilities when you need them.

The future of local AI

Local AI is not a niche hobby — it is becoming the default way to run AI for an increasing number of use cases. Several trends are accelerating this shift:

- **Model efficiency is improving faster than model size is growing**. Techniques like mixture-of-experts (MoE), speculative decoding, and better training data mean smaller models keep getting more capable. A 2026 7B model outperforms a 2023 70B model on most benchmarks.
- **Hardware is getting cheaper and more capable**. Each generation of consumer GPUs and Apple Silicon chips offers significantly more memory and bandwidth. NVIDIA's consumer GPUs now come with up to 24 GB VRAM, and Apple's M-series chips offer up to 192 GB unified memory.
- **Quantization keeps improving**. New techniques like AQLM, QuIP#, and HQQ push the quality of 2-4 bit quantized models closer to full precision, meaning you can run larger models on the same hardware.
- **The tooling is maturing**. Projects like Ollama, Open WebUI, and LM Studio have made local AI genuinely easy to use. The gap between "install ChatGPT" and "install local AI" is shrinking to zero.
- **Regulations are tightening**. GDPR, the EU AI Act, HIPAA enforcement, and similar regulations worldwide are making data locality not just a preference but a requirement for many organizations.

Local AI is not about rejecting cloud services. It is about having the choice — the technical capability and the practical tools — to run AI wherever it makes sense. For many tasks, that place is your own machine.

Ready to get started? Read Why Run AI Locally? for the full case, or jump straight to our quickstart guide to have a model running in five minutes.

Frequently Asked Questions

What is local AI? +

Do I need a powerful GPU to run AI locally? +

Is local AI as good as ChatGPT? +

What tools do I need to run AI locally? +

Is my data safe with local AI? +

local-llm.net

Independent guidance and accountable delivery for private AI: on your devices or infrastructure, with measured quality, performance and cost.

A **Neul Labs** property.

contact@local-llm.net

© 2026 Neul Labs Limited. Registered in Scotland, company no. SC767862. All rights reserved.